



Programación

Interfaces gráficas: Swing II

(Maquetación y escuchadores)

El gestor de maquetación (*layout manager*)

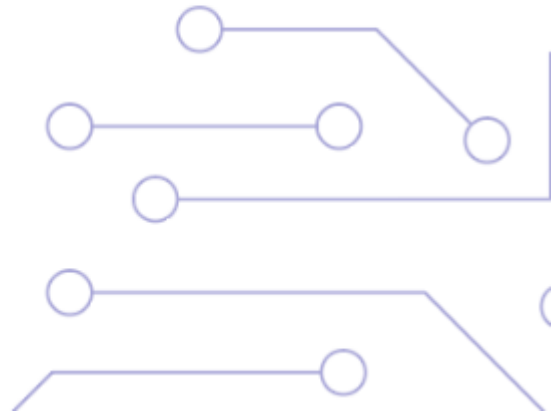
- El gestor de maquetación especifica cómo deben posicionarse los elementos dentro de un contenedor.
 - Aunque se puede especificar el tamaño y la posición de los elementos, el gestor de maquetación tiene la última palabra.
- En concreto, se deberá determinar el contenedor (*content pane*) de los Jframes y la maquetación de los Jpanels creados.
 - Por defecto, los contenedores usan el gestor de maquetación *FlowLayout* y los Jpanels usan el gestor de maquetación *BorderLayout*.
- Si se coloca otro contenedor dentro de un contenedor, puede usar un gestor de maquetación diferente.
 - Esto permite la creación de aplicaciones más complejas.

El gestor de maquetación (Layout manager)

- El gestor de maquetación de un contenedor se puede asignar en el constructor o utilizando el método `setLayoutManager()`.

```
JPanel panel = new JPanel(new BorderLayout());
```

```
Container contentPane = frame.getContentPane();  
contentPane.setLayout(new FlowLayout());
```



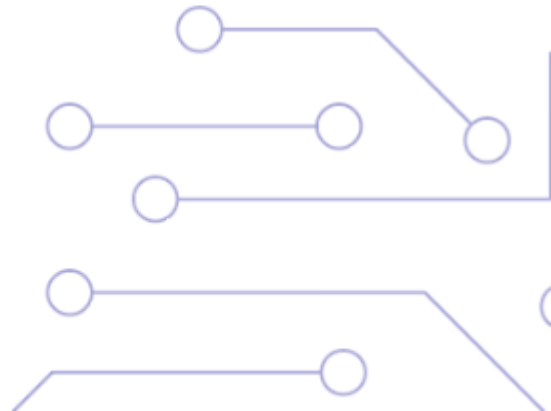
Agregar elementos a un contenedor

- Como ya hemos visto, los elementos se añaden a un contenedor mediante el método *add()*.
- Los parámetros de este método dependen del gestor de maquetación utilizado.
 - Por ejemplo, en el gestor *GroupLayout* no es necesario añadir elementos explícitamente.
 - En otro ejemplo, en el gestor *BorderLayout* es necesario indicar en qué área se añadirá el elemento mediante una de las constantes proporcionadas para este fin.

```
panel.add(elemento, BorderLayout.PAGE_START);
```

FlowLayout

- El gestor de maquetación más sencillo.
- Coloca los elementos horizontalmente, comenzando desde la esquina superior izquierda hasta el borde del contenedor y luego pasa a la siguiente línea.
- Intenta mantener el tamaño especificado para cada elemento.
- Si el tamaño del contenedor es mayor que el de los elementos que contiene, los centra.
 - La alineación se puede especificar en el constructor.





FlowLayout

- Constructores:
 - FlowLayout()
 - FlowLayout(int alineacion)
 - FlowLayout.LEFT
 - FlowLayout.CENTER
 - FlowLayout.RIGHT
 - FlowLayout.LEADING
 - FlowLayout.TRAILING
 - FlowLayout(int alineación, int hesp, int besp)
- Métodos: void setHGap(int), void setVGap(int), void setAligment(int alignement)

FlowLayout

```
MiVentana()  
{  
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    setPreferredSize(new Dimension(400, 300));  
    setLayout(new FlowLayout());  
    add(new JButton("Botón 1"));  
    add(new JButton("Botón 2"));  
    add(new JButton("Botón 3"));  
    add(new JButton("Botón 4"));  
    add(new JButton("Botón 5"));  
    pack();  
    setVisible(true);  
}
```

FlowLayout

```
Miventana()  
{  
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    setPreferredSize(new Dimension(400, 300));  
    FlowLayout f = new FlowLayout();  
    f.setAlignment(FlowLayout.RIGHT);  
    f.setHgap(20);  
    setLayout(f);  
    add(new JButton("Botón 1"));  
    add(new JButton("Botón 2"));  
    add(new JButton("Botón 3"));  
    add(new JButton("Botón 4"));  
    add(new JButton("Botón 5"));  
    pack();  
    setVisible(true);  
}
```

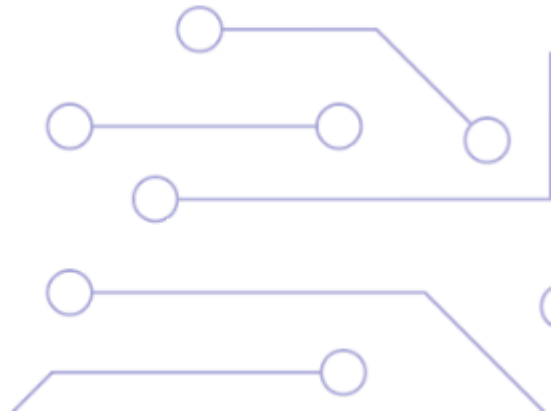



BorderLayout

- Divide el contenedor en 5 áreas, cada una de las cuales puede contener solo un elemento o contenedor.
 - Áreas: N (norte), S (sur), E (este), W (oeste) y C (centro).
 - El centro ocupa todo el espacio posible.
 - Las demás áreas ocupan el espacio mínimo necesario para mostrar los elementos que contienen.
- Al añadir un elemento o contenedor, se debe indicar en qué área se añade.

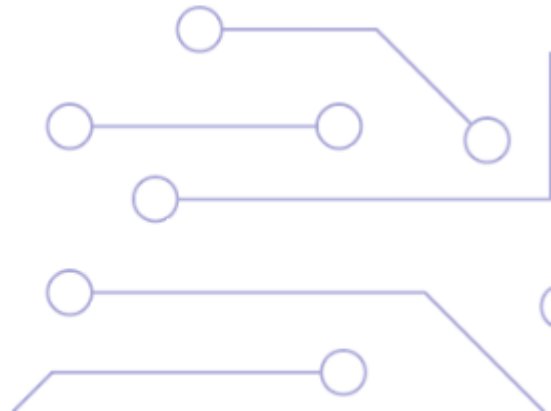
BorderLayout

- Si un elemento o contenedor se ubica al norte o al sur, el gestor respetará su altura, pero intentará reducir su ancho.
- Por el contrario, si un elemento o contenedor se ubica al este o al oeste, el gestor respetará su ancho, pero intentará reducir su altura.
- Un elemento o contenedor ubicado en el centro ocupará el máximo espacio posible.



BorderLayout

- Constructores:
 - BorderLayout()
 - BorderLayout(int hesp, int besp)
- Métodos:
 - void setHGap(int)
 - void setVGap(int)
 - void setAlignment(int alineamiento)





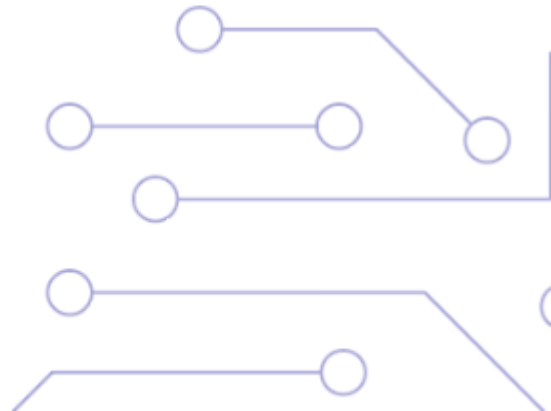
BorderLayout

```
MiVentana()  
{  
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    setLayout(new BorderLayout());  
    add(new JButton("Norte"), BorderLayout.NORTH);  
    add(new JButton("Sur"), BorderLayout.SOUTH);  
    add(new JButton("Este"), BorderLayout.EAST);  
    add(new JButton("Oeste"), BorderLayout.WEST);  
    add(new JButton("Centro"), BorderLayout.CENTER);  
    pack();  
    setVisible(true);  
}
```



GridLayout

- Divide el contenedor en celdas del mismo tamaño y coloca un elemento en cada una.
 - Comienza a llenar de izquierda a derecha y de arriba a abajo.
 - El número de filas y columnas se especifica en el constructor.
- Constructores:
 - GridLayout(int cantLinea, int cantColumnas)
 - GridLayout(int cantLinea, int cantColumnas, int hesp, int besp)
- Métodos:
 - void setHGap(int)
 - void setVGap(int)
 - void setRows(int)
 - void setColumns(int)



GridLayout

```
MiVentana()  
{  
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    setLayout(new GridLayout(2,3));  
    add(new JButton("Botón 1"));  
    add(new JButton("Botón 2"));  
    add(new JButton("Botón 3"));  
    add(new JButton("Botón 4"));  
    add(new JButton("Botón 5"));  
    pack();  
    setVisible(true);  
}
```

GridLayout

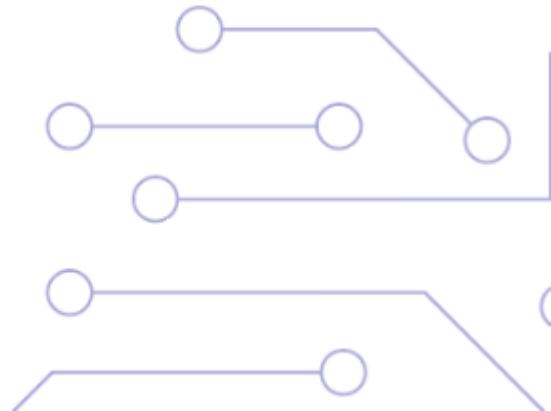
```
MiVentana()  
{  
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    setLayout(new GridLayout(3, 0, 20, 20));  
    add(new JButton("Botón 1"));  
    add(new JButton("Botón 2"));  
    add(new JButton("Botón 3"));  
    add(new JButton("Botón 4"));  
    add(new JButton("Botón 5"));  
    pack();  
    setVisible(true);  
}
```



Si se especifica 0, se tomarán los parámetros requeridos junto con los demás parámetros completados.

BoxLayout

- Organiza los elementos o contenedores en una sola fila (eje x) o una sola columna (eje y).
 - La fila o columna se ajusta al tamaño del contenedor.
- Cuando se establece en horizontal, intenta respetar el ancho de los elementos, y cuando se establece en vertical, intenta respetar la altura de los elementos.
- Constructor:
 - `BoxLayout(Container container, int eje)`
 - `BoxLayout.X_AXIS` o `BoxLayout.Y_AXIS`



BoxLayout

```
MiVentana()  
{  
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    setLayout(new BoxLayout(getContentPane(), BoxLayout.X_AXIS));  
    JPanel panel1 = new JPanel();  
    panel1.setBorder(BorderFactory.createTitledBorder("Panel1"));  
    JPanel panel2 = new JPanel();  
    panel2.setBorder(BorderFactory.createTitledBorder("Panel2"));  
    panel1.setLayout(new BoxLayout(panel1, BoxLayout.Y_AXIS));  
    panel2.setLayout(new BoxLayout(panel2, BoxLayout.Y_AXIS));  
    for (int i = 0; i < 3; i++) {  
        panel1.add(new JButton("Botón " + i));  
        panel2.add(new JButton("Botón " + i));  
    }  
    add(panel1);  
    add(panel2);  
    setSize(200,300);  
    setTitle("BoxLayout");  
    pack();  
    setVisible(true);  
}
```

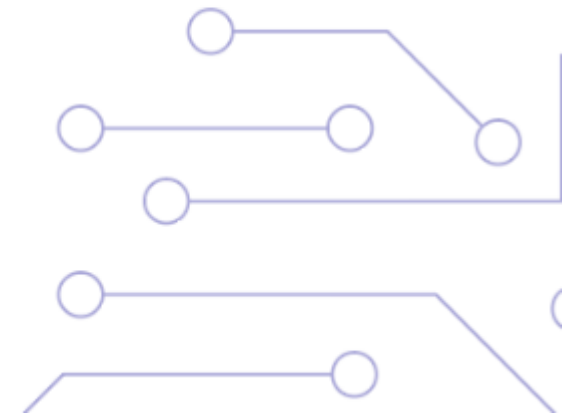
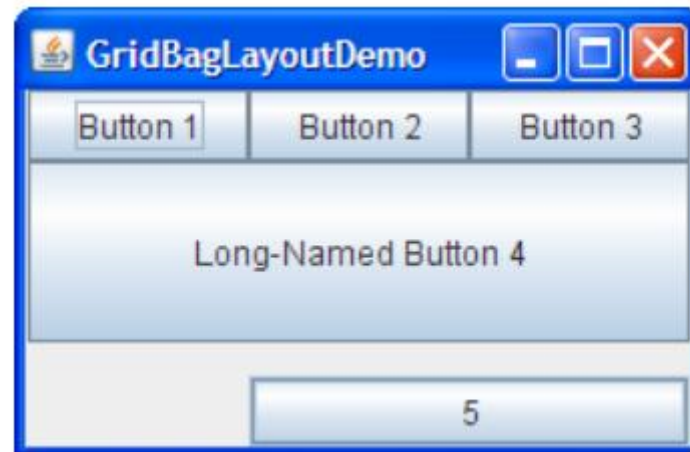
CardLayout

- Permite mostrar diferentes conjuntos de elementos en distintos momentos.
 - Normalmente, las opciones de un *JComboBox* se utilizan para seleccionar el conjunto de elementos que se mostrarán.
- Un sistema de maquetación similar podría ser usar un *JTabbedPane*.



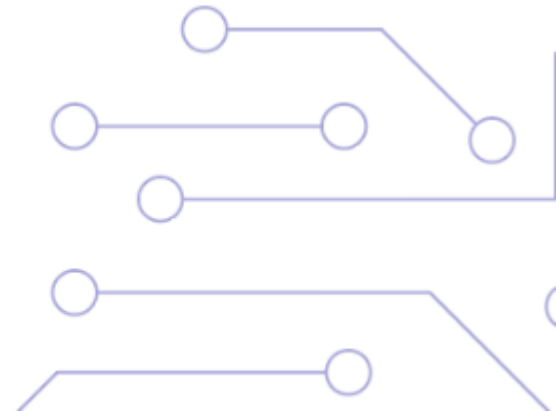
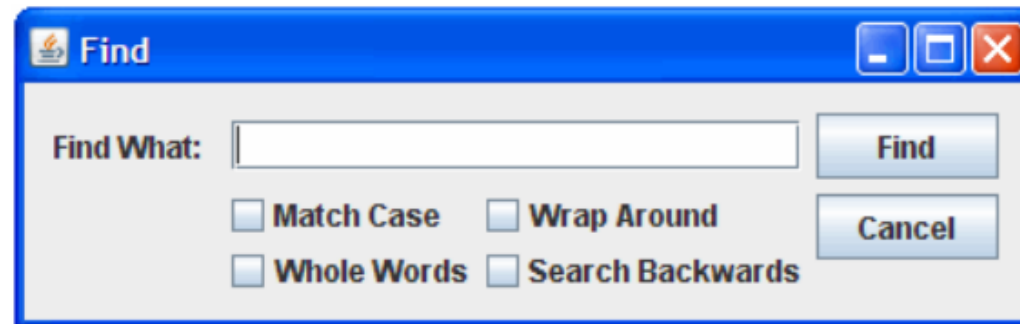
GridBagLayout

- Es un gestor de maquetación complejo pero muy flexible.
- El contenedor está dividido en celdillas, pero los elementos que contiene pueden ocupar más de una.
- Además, las filas pueden tener diferentes alturas y las columnas, diferentes anchos.



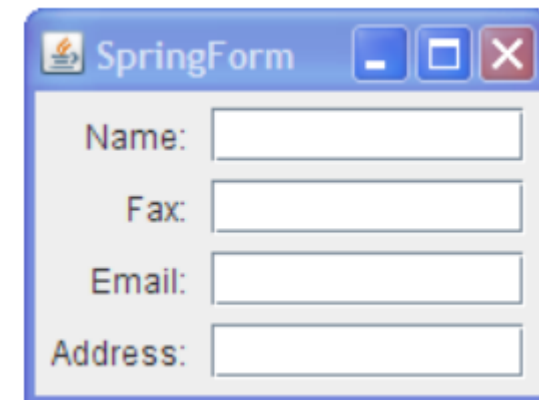
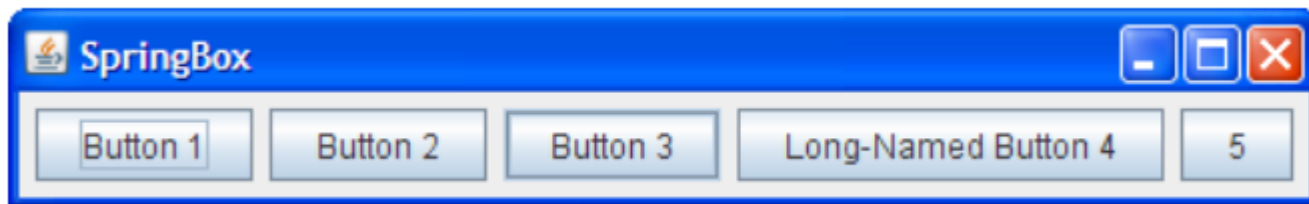
GroupLayout

- Es un gestor de maquetación que puede ser bastante complejo, pero potente.
- Fue diseñado para funcionar gráficamente, pero también puede usarse en programación manual.
- La maquetación se define de forma independiente para los ejes horizontal y vertical.
 - Cada elemento tiene dos maquetaciones.



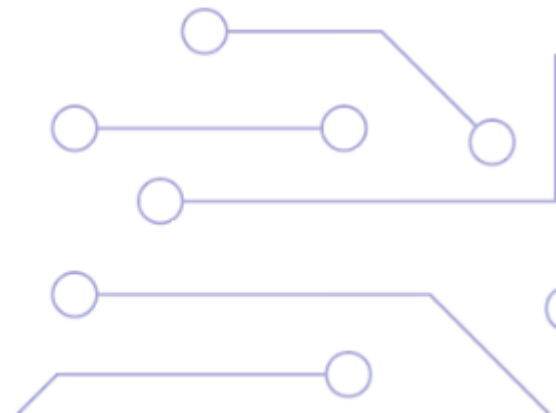
SpringLayout

- Es un gestor de maquetación flexible creado para uso gráfico.
- Consiste en establecer restricciones de distancia entre elementos dentro de un contenedor.
 - Estas restricciones de distancia se pueden calcular dinámicamente.



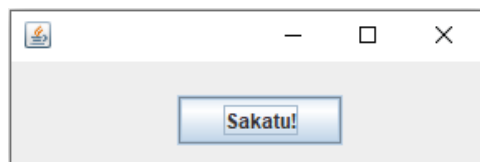
Eventos y escuchadores

- En una aplicación gráfica, el flujo de ejecución se basa en las acciones que el usuario realiza en la interfaz.
 - Estas acciones se denominan eventos (**events**).
 - Cada vez que se pulsa una tecla, se mueve el ratón o se hace clic, se activa un evento.
- Estos eventos se pueden definir con antelación para que, al activarse, el programa los "escuche" y tome las medidas adecuadas.

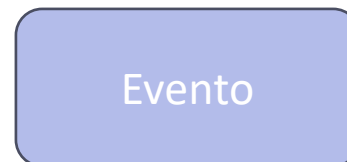
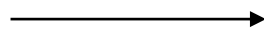


Eventos y escuchadores

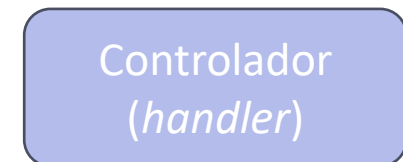
- Por ejemplo, cuando el usuario presiona un botón, se debe ejecutar un método específico.
- Para ello:
 - Hay que identificar el botón que puede activar el evento.
 - Cuando se activa el evento hay que describir las acciones que deben realizarse.



El usuario presiona el botón



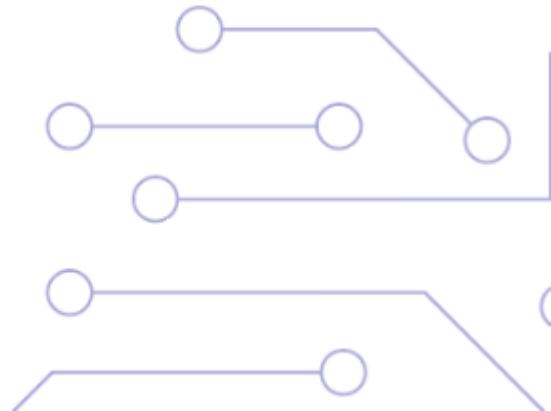
El sistema activa el evento.



El controlador recibe la información del evento y ejecuta el método.

Eventos y escuchadores

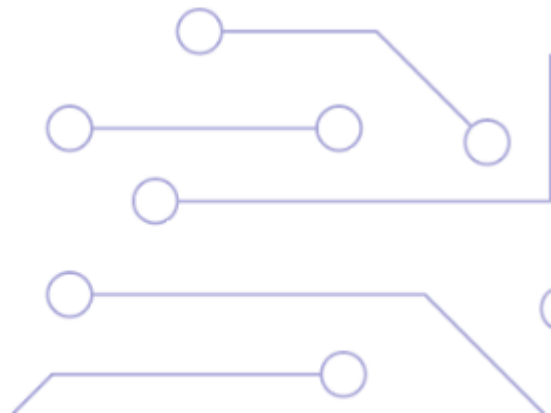
- Se deben tener en cuenta tres elementos para que se active un evento:
 - Origen del evento: Determina en qué elementos de la interfaz gráfica ocurre.
 - Escuchador del evento (**listener**): Proceso que espera a que se active el evento.
 - Gestor del evento (**handler**): Determina qué hacer cuando se activa el evento.



Eventos y escuchadores

- Origen del evento:
 - Es el elemento de la interfaz gráfica que activa el evento.
 - Este elemento está asociado a un escuchador de eventos.
 - Suponiendo que un evento se active al presionar un botón, el código sería lo siguiente:
 - Esto indica que se puede activar un evento en ese botón y esperará a que ocurra.

```
 JButton boton = new JButton("¡Pulsar!");  
 boton.setBounds(...);  
 boton.addActionListener(...);  
 add(boton);
```



Eventos y escuchadores

- El escuchador de eventos (**listener**):
 - Este es el elemento que escucha (o recibe) cuando se activa un evento.
 - Puede ser cualquier objeto que implemente una de las interfaces que Java proporciona para escuchar eventos.
 - Estas son las interfaces más utilizadas para escuchar eventos.
 - Dependiendo del elemento que se active, algunos estarán disponibles y otros no.

Interfaz	¿Qué evento espera?
ActionListener	Varía según el elemento, así en un botón es hacer clic en este, en un cuadro de texto es presionar la tecla enter y en un ComboBox es seleccionar un elemento.
WindowListener	Minimizar, ampliar, mover una ventana...
MouseListener	Hacer clic o soltar un botón del ratón sobre un elemento

Eventos y escuchadores

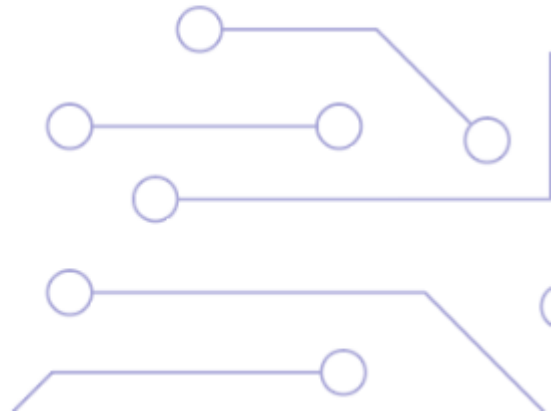
- El escuchador de eventos(*listener*):

Interfaz	¿Qué evento espera?
MouseMotionListener	Pasar el cursor sobre un elemento
ComponentListener	Mostrar, ocultar, redimensionar un elemento...
FocusListener	Que un elemento coja o pierda el foco del teclado
TextListener	Cambiar el texto en un cuadro de texto
KeyListener	Presionar o soltar una tecla
ItemListener	Cambiar el estado de un Checkbox o seleccionar un elemento en una List o en un ComboBox
ListSelectionListener	Seleccionar un elemento en una List o en una Table

Eventos y escuchadores

- Hay cuatro maneras diferentes de implementar escuchadores:
 - 1. La ventana realiza el trabajo de escucha.
 - Siguiendo con el ejemplo anterior, la ventana debe implementar la interfaz ActionListener.

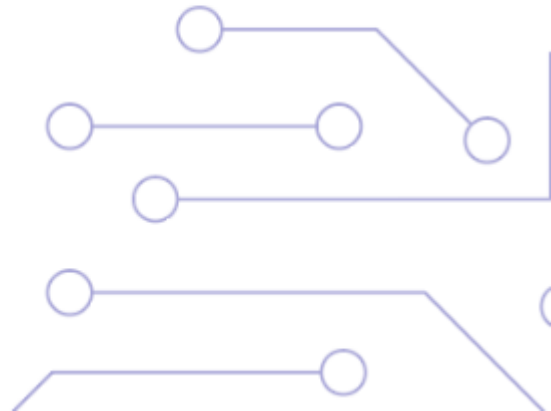
```
public class MiVentana extends JFrame implements ActionListener
{
    MiVentana()
    {
        ...
        JButton boton = new JButton("¡Pulsar!!");
        boton.addActionListener(this);
        add(boton);
        ...
    }
}
```



Eventos y escuchadores

- La implementación de escuchadores se puede realizar de cuatro maneras diferentes.
 - 2. Se crea una clase para realizar la función de escuchador.
 - La nueva clase creada debe implementar la interfaz ActionListener.

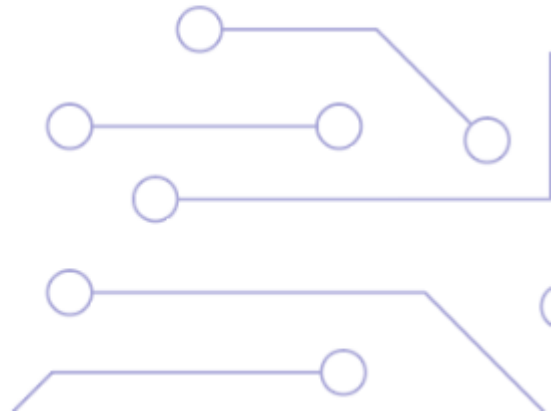
```
public class MiVentana extends JFrame
{
    MiVentana()
    {
        ...
        JButton boton = new JButton("¡Pulsar!");
        boton.addActionListener(new BotonEscuchador ());
        add(boton);
        ...
    }
    public class BotonEscuchador implements ActionListener{}
```



Eventos y audiencias

- La implementación de los escuchadores se puede realizar de cuatro maneras diferentes.
 - 3. Se crea una clase interna para realizar el trabajo del escuchador.
 - Similar al ejemplo anterior, pero la clase se crea internamente.
 - Esto ofrece ventajas cuando se necesita acceder a una característica de la clase (lo cual es frecuente).

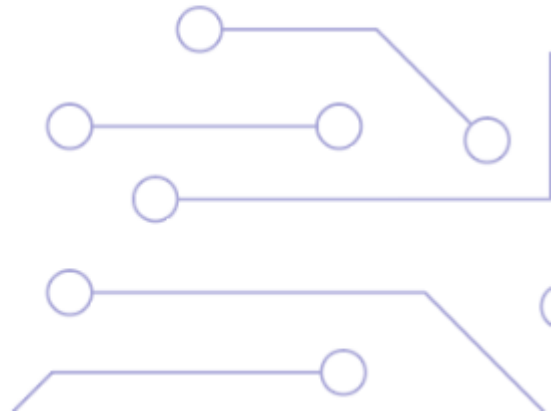
```
public class MiVentana extends JFrame
{
    MiVentana()
    {
        ...
        JButton boton = new JButton("¡Pulsar!");
        boton.addActionListener(new BotonEscuchador());
        add(boton);
        ...
    }
    public class BotonEscuchador implements ActionListener{}
}
```



Eventos y escuchadores

- La implementación de los escuchadores se puede realizar de cuatro maneras diferentes.
 - 4. Se crea una clase interna anónima para realizar el trabajo del escuchador.
 - Aunque pueda parecer mal implementada, este comando nos indica que debemos crear una clase que implemente esta interfaz y escribir la implementación en ella.

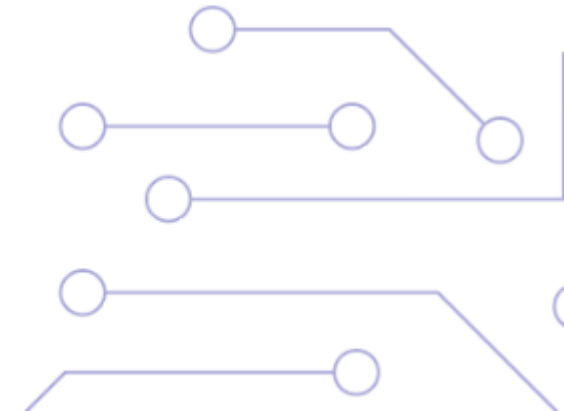
```
public class MiVentana extends JFrame
{
    MiVentana()
    {
        ...
        JButton boton = new JButton("¡Pulsar!");
        boton.addActionListener(new ActionListener {...}));
        add(boton);
        ...
    }
}
```



Eventos y escuchadores

- Controlador de eventos (*handler*):
 - Es el método que debe ejecutarse cuando se activa un evento.
 - Consiste en implementar uno o más de los métodos definidos en la interfaz del escuchador.
 - Los escuchadores de tipo XXXListener tienen los siguientes métodos:

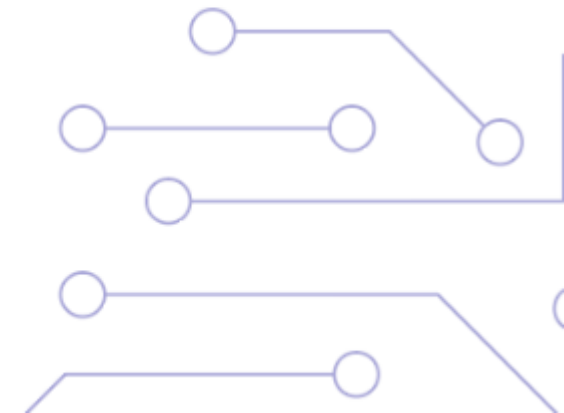
Interfaz	Métodos
ActionListener	actionPerformed (ActionEvent)
WindowListener	windowActivated (WindowEvent) windowClosed (WindowEvent) windowOpened (WindowEvent) ...



Eventos y escuchadores

- Controlador de eventos (*handler*):
 - Los escuchadores de tipo XXXListener tienen los siguientes métodos:

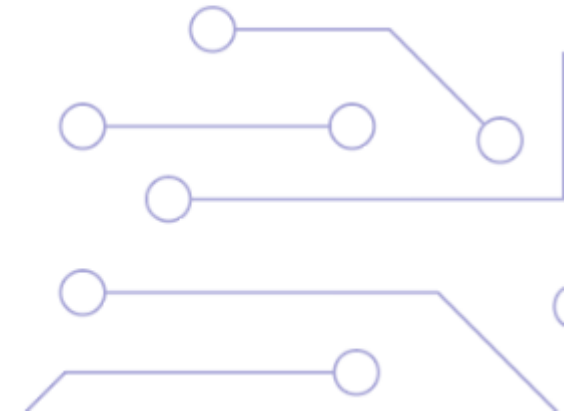
Interfaz	Métodos
MouseListener	mouseClicked (MouseEvent) mousePressed (MouseEvent) mouseReleased (MouseEvent) mouseEntered (MouseEvent) mouseExited (MouseEvent)
MouseMotionListener	mouseMoved (MouseEvent) mouseDragged (MouseEvent)
ComponentListener	componentHidden (ComponentEvent) componentMoved (ComponentEvent) componentResized (ComponentEvent) componentShown (ComponentEvent)



Eventos y escuchadores

- Controlador de eventos (*handler*):
 - Los escuchadores de tipo XXXListener tienen los siguientes métodos:

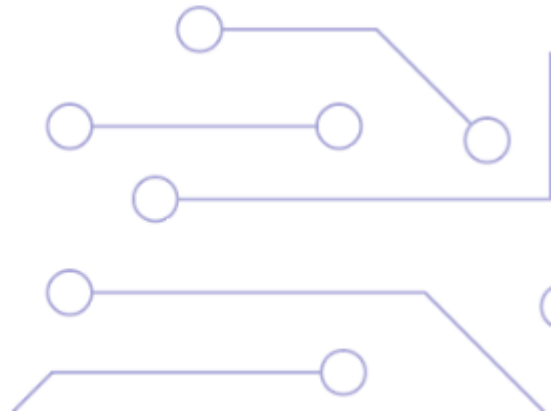
Interfaz	Métodos
FocusListener	focusGained (FocusEvent) focusLost (FocusEvent)
TextListener	textValueChanged (TextEvent)
KeyListener	keyPressed (KeyEvent) keyReleased (KeyEvent) keyTyped (KeyEvent)
ItemListener	itemStateChanged(ItemEvent)



Eventos y escuchadores

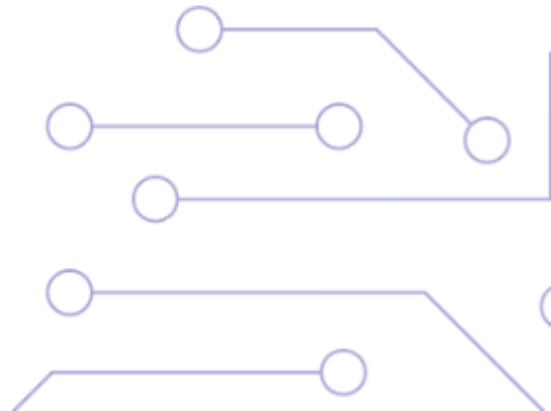
- Suponiendo que la ventana actúa como escuchador (caso 1), la clase de la ventana también debe implementar todos los métodos del escuchador.
 - En el ejemplo mostrado, solo un método, *ActionPerformed()*

```
public class MiVentana extends JFrame implements ActionListener
{
    MiVentana()
    {
        ...
        boton.addActionListener(this);
        ...
    }
    public void actionPerformed(ActionEvent e){}
}
```



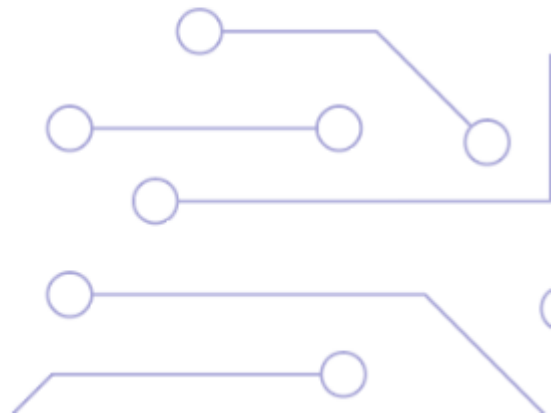
Las clases *XXXEvent*

- Los métodos de las interfaces de los escuchadores que deben implementarse reciben objetos de este tipo como parámetros.
 - *ActionEvent* en el ejemplo anterior.
- Estos objetos proporcionan más información sobre el evento.
 - Por ejemplo, la clase *KeyEvent* tiene un método *getKeyChar()* que devuelve el valor de la tecla pulsada.
- Existe un método muy útil que todas las clases de este tipo tienen: *Object getSource()*.
 - Este método devuelve el objeto que activó el evento.



Los controladores compartidos

- Es posible usar el mismo controlador para gestionar eventos de múltiples elementos.
 - Por ejemplo, una ventana con dos botones siendo la propia ventana escuchador.
 - El escuchador es el mismo.
 - El controlador es único.
 - Cada botón ejecutará una acción diferente.



Los controladores compartidos

```
public class MiVentana extends JFrame implements ActionListener
{
    JButton botonRojo, botonAzul;
    MiVentana(){
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(null);
        setPreferredSize(new Dimension(300, 200));
        botonRojo = new JButton("Rojo");
        botonRojo.setBounds(50,50,80,30);
        botonRojo.addActionListener(this);
        add(botonRojo);
        botonAzul = new JButton("Azul");
        botonAzul.setBounds(150,50,80,30);
        botonAzul.addActionListener(this);
        add(botonAzul);
        pack();
        setVisible(true);
    }
}
```

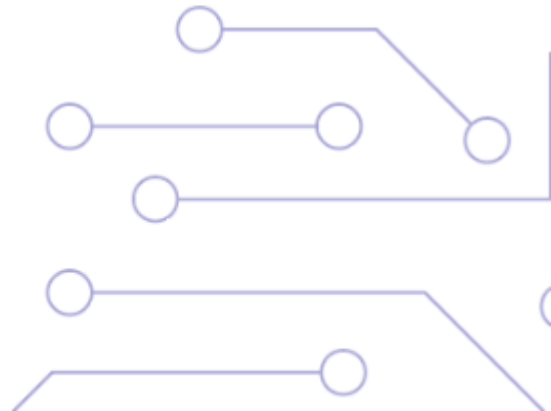
Los controladores compartidos

```
public class MiVentana extends JFrame implements ActionListener
{
    ...
    public void actionPerformed(ActionEvent e)
    {
        JButton botonPulsado = (JButton)e.getSource();
        if(botonPulsado.equals(botonRojo))
            this.getContentPane().setBackground(Color.RED);
        if (botonPulsado.equals(botonAzul))
            this.getContentPane().setBackground(Color.BLUE);
    }
}
```

Las clases adaptadoras

- La mayoría de los escuchadores tienen más de un método.
 - Incluso si no utiliza algunos de estos métodos, es necesario implementarlos.

```
public class EscuchadorRaton implements MouseListener
{
    public void mousePressed(MouseEvent e) {
    }
    public void mouseReleased(MouseEvent e) {
    }
    public void mouseEntered(MouseEvent e) {
    }
    public void mouseExited(MouseEvent e) {
    }
    public void mouseClicked(MouseEvent e) {
        // Acciones cuando se hace clic con el ratón
    }
}
```



Las clases adaptadoras

- El ejemplo anterior es correcto; los métodos pueden dejarse sin cuerpo, pero no es apropiado.
- Para evitar esto, existen clases *XXXAdapter*, una para cada interfaz de escucha que tenga más de un método.
 - Clase *MouseAdapter* para la interfaz *MouseListener*.
 - Porque la interfaz *ActionListener* no tiene un solo método.
- Las clases adaptadoras son clases que implementan métodos de interfaz vacíos.
 - Queda más limpio extenderr estas clases y solo sobrescribir los métodos necesarios que implementar la interfaz y dejar muchos métodos vacíos.

Las clases adaptadoras

- Por ejemplo, supongamos que solo nos interesa el evento de clic del ratón.
 - La clase escuchadora extenderá la clase *MouseAdapter*, de modo que se implementarán todos los métodos de la interfaz *MouseListener*.
 - Después la clase escuchadora reescribirá el método *mouseClicked()* que nos interesa para hacer lo que queremos que haga al hacer clic con el ratón.

```
public class EscuchadorRaton extends MouseAdapter
{
    public void mouseClicked(MouseEvent e)
    {
        // Acciones cuando se hace clic con el raton
    }
}
```

Las clases adaptadoras

- ¿Qué se puede hacer si una clase ya extiende otra (normalmente JFrame)?
 - Solución: Usar **una clase Interna** o una clase interna anónima.

```
public class MiVentana extends JFrame
{
    MiVentana() {
        ...
        boton.addMouseListener(new EscuchadorRaton());
    }
    public class EscuchadordeRaton extends MouseAdapter {
        public void mouseClicked(MouseEvent e) {
            // Acciones cuando se hace clic con el raton
        }
    }
}
```

Las clases adaptadoras

- ¿Qué se puede hacer si una clase ya extiende otra (normalmente JFrame)?
 - Solución: Usar una clase interna o una clase interna anónima.

```
public class MiVentana extends JFrame
{
    MiVentana() {
        ...
        boton.addMouseListener(new MouseAdapter() {
            public void mouseClicked(MouseEvent e) {
                // Acciones cuando se hace clic con el raton
            }
        });
    }
}
```